

Pandas is a fast, flexible, and expressive Python library

It provides data structures and functions to efficiently manipulate structured (tabular), time-series, and labeled data.

Main data structures:

Series: 1D labeled array

DataFrame: 2D labeled table (rows and columns)

Benefits of Pandas

Simple syntax Easy to read and write code

Fast and efficient

Powerful table-like structure

Handle CSV, Excel, SQL, JSON, etc.

Filtering, slicing, conditional logic

install pandas

import pandas as pd

```
import pandas as pd
import numpy as np
```

```
s = pd.Series([10, 20, 30, np.nan, 50])
print(s)
```

```
↔ 0    10.0
   1    20.0
   2    30.0
   3     NaN
   4    50.0
   dtype: float64
```

```
# Series
s = pd.Series([10, 20, 30, np.nan, 50])
print(s)
```

```
# DataFrame from dictionary
data = {
```

```

    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Salary': [50000, 60000, 70000]
}
df = pd.DataFrame(data)
print(df)

```

```

⇒ 0    10.0
   1    20.0
   2    30.0
   3     NaN
   4    50.0
dtype: float64
      Name  Age  Salary
0    Alice   25   50000
1     Bob   30   60000
2  Charlie   35   70000

```

```

# Save to CSV
df.to_csv("employees5555.csv", index=False)

```

```

# Read from CSV
df2 = pd.read_csv("employees5555.csv")
print(df2)

```

```

⇒      Name  Age  Salary
0    Alice   25   50000
1     Bob   30   60000
2  Charlie   35   70000

```

```

# Series
s = pd.Series([10, 20, 30, np.nan, 50])
print(s)

```

```

# DataFrame from dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Salary': [50000, 60000, 70000]
}
df = pd.DataFrame(data)
print(df)

```

```

⇒ 0    10.0
   1    20.0
   2    30.0
   3     NaN
   4    50.0
dtype: float64
      Name  Age  Salary

```

```

0    Alice    25    50000
1     Bob    30    60000
2  Charlie    35    70000

```

```

# Save to CSV
df.to_csv("m5.csv")

# Read from CSV
df2 = pd.read_csv("m5.csv")
print(df2)

```

```

➡ Unnamed: 0    Name  Age  Salary
0         0    Alice   25   50000
1         1     Bob   30   60000
2         2  Charlie   35   70000

```

```

print(df['Name'])          # Select column
print(df[['Name', 'Salary']]) # Multiple columns
print(df[df['Age'] > 28])  # Conditional filter

```

```

➡ 0    Alice
1    Bob
2  Charlie
Name: Name, dtype: object
   Name  Salary
0  Alice  50000
1    Bob  60000
2  Charlie 70000
   Name  Age  Salary
1    Bob   30  60000
2  Charlie  35  70000

```

```
df['Bonus'] = np.array([1000, 2000, 1500]) # Add new column
```

df

```

➡

```

	Name	Age	Salary	Bonus
0	Alice	25	50000	1000
1	Bob	30	60000	2000
2	Charlie	35	70000	1500

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
df.drop('Bonus', axis=1, inplace=False)    # Delete column
```



	Name	Age	Salary
0	Alice	25	50000
1	Bob	30	60000
2	Charlie	35	70000



df



	Name	Age	Salary	Bonus
0	Alice	25	50000	1000
1	Bob	30	60000	2000
2	Charlie	35	70000	1500



Next steps:

[Generate code with df](#)[View recommended plots](#)[New interactive sheet](#)

```
df_read = pd.read_excel("employees.xlsx")
print("\nData Read from Excel:\n", df_read)
```



Data Read from Excel:

	ID	Name	Department	Salary
0	1	Alice	HR	50000
1	2	Bob	IT	60000
2	3	Charlie	Finance	55000

```
df_json = pd.read_json("data.json")
print(df_json)
```



	id	name	age	city	salary
0	1	Alice	25	New York	70000
1	2	Bob	30	Los Angeles	85000
2	3	Charlie	28	Chicago	62000
3	4	David	35	Houston	91000
4	5	Eva	22	Phoenix	55000

```
s = pd.Series([1, None, 3, None, 5])
print(s.fillna(0))    # Replace NaN with 0
print(s.dropna())     # Drop NaN
```



```
0    1.0
1    0.0
2    3.0
```

```
3    0.0
4    5.0
dtype: float64
0    1.0
2    3.0
4    5.0
dtype: float64
```

df



	Name	Age	Salary	Bonus	
0	Alice	25	50000	1000	
1	Bob	30	60000	2000	
2	Charlie	35	70000	1500	

Next steps:

[Generate code with df](#)[View recommended plots](#)[New interactive sheet](#)

```
df['Salary x 2'] = df['Salary'].apply(lambda x: x * 2)
```

df



	Name	Age	Salary	Bonus	Salary x 2	
0	Alice	25	50000	1000	100000	
1	Bob	30	60000	2000	120000	
2	Charlie	35	70000	1500	140000	

Next steps:

[Generate code with df](#)[View recommended plots](#)[New interactive sheet](#)

```
# Create sample data
data = {
    'Dept': ['HR', 'HR', 'IT', 'IT', 'Admin'],
    'Salary': [30000, 35000, 40000, 42000, 28000]
}
df = pd.DataFrame(data)
```

df

	Dept	Salary	
0	HR	30000	
1	HR	35000	
2	IT	40000	
3	IT	42000	
4	Admin	28000	

Next steps:

[Generate code with df](#)
[View recommended plots](#)
[New interactive sheet](#)

```
# Group and aggregate
print(df.groupby('Dept').mean())
```

	Salary
Dept	
Admin	28000.0
HR	32500.0
IT	41000.0

```
df1 = pd.DataFrame({'ID': [1, 2], 'Name': ['Alice', 'Bob']})
df2 = pd.DataFrame({'ID': [1, 2], 'Score': [95, 88]})
merged = pd.merge(df1, df2, on='ID')
print(merged)
```

	ID	Name	Score
0	1	Alice	95
1	2	Bob	88

```
import pandas as pd
import sqlite3
```

```
# Step 1: Create sample DataFrame
data = {
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [23, 25, 22],
    'City': ['Delhi', 'Mumbai', 'Bangalore']
}
df = pd.DataFrame(data)
```

```
# Step 2: Create SQLite connection (file-based)
conn = sqlite3.connect('students.db')
```

```
# Step 3: Write DataFrame to SQL table
df.to_sql('students', conn, if_exists='replace', index=False)
```

```
# Step 4: Read data back from SQL table
df_read = pd.read_sql_query('SELECT * FROM students', conn)
print(df_read)
```

```
# Step 5: Close the connection
conn.close()
```

```
⇒
```

	ID	Name	Age	City
0	1	Alice	23	Delhi
1	2	Bob	25	Mumbai
2	3	Charlie	22	Bangalore

```
import pandas as pd
```

```
mydataset = {
    'cars': ["BMW", "Volvo", "Ford"],
    'passings': [3, 7, 2]
}
```

```
myvar = pd.DataFrame(mydataset)
```

```
print(myvar)
```

```
⇒
```

	cars	passings
0	BMW	3
1	Volvo	7
2	Ford	2

```
pd.__version__
```

```
⇒ '2.2.2'
```

```
import pandas as pd
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a)
```

```
print(myvar)
```

```
⇒
```

0	1
1	7
2	2

dtype: int64

```
print(myvar[0])
```

```
⇒ 1
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a, index = ["x", "y", "z"])
```

```
print(myvar)
```

```
⇒ x    1  
   y    7  
   z    2  
   dtype: int64
```

```
print(myvar["y"])
```

```
⇒ 7
```

```
calories = {"day1": 420, "day2": 380, "day3": 390}
```

```
myvar = pd.Series(calories)
```

```
print(myvar)
```

```
⇒ day1    420  
   day2    380  
   day3    390  
   dtype: int64
```

```
calories = {"day1": 420, "day2": 380, "day3": 390}
```

```
myvar = pd.Series(calories, index = ["day1"])
```

```
print(myvar)
```

```
⇒ day1    420  
   dtype: int64
```

```
data = {  
    "calories": [420, 380, 390],  
    "duration": [50, 40, 45]  
}
```

```
myvar = pd.DataFrame(data)
```

```
print(myvar)
```

```
⇒   calories  duration  
0         420         50  
1         380         40  
2         390         45
```



```
myvar.loc[0]
```

```
⇒
```

	0
calories	420
duration	50

dtype: int64

```
myvar.loc[0,'duration']
```

```
⇒ np.int64(50)
```

```
print(myvar.loc[[0, 1]])
```

```
⇒
```

	calories	duration
0	420	50
1	380	40

```
data = {  
    "calories": [420, 380, 390],  
    "duration": [50, 40, 45]  
}
```

```
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])
```

```
print(df)
```

```
⇒
```

	calories	duration
day1	420	50
day2	380	40
day3	390	45

```
print(df.loc["day2"])
```

```
⇒
```

calories	380
duration	40

Name: day2, dtype: int64

```
df = pd.read_csv('imdb_data.csv')
```

```
df
```



	id	belongs_to_collection		budget	genres	homepage
0	1	['id': 313576, 'name': 'Hot Tub Time Machine ...		14000000	['id': 35, 'name': 'Comedy']	NaN
1	2	['id': 107674, 'name': 'The Princess Diaries ...		40000000	['id': 35, 'name': 'Comedy'], {'id': 18, 'nam...	NaN
2	3	NaN	NaN	3300000	['id': 18, 'name': 'Drama']	http://sonyclassics.com/whiplash/
3	4	NaN	NaN	1200000	['id': 53, 'name': 'Thriller'], {'id': 18, 'n...	http://kahaanithefilm.com/
4	5	NaN	NaN	0	['id': 28, 'name': 'Action'], {'id': 53, 'nam...	NaN
...	...	...	...	...	...	...
2995	2996	NaN	NaN	0	['id': 35, 'name': 'Comedy'], {'id': 10749, '...	NaN
2996	2997	NaN	NaN	0	['id': 18, 'name': 'Drama'], {'id': 10402, 'n...	NaN
2997	2998	NaN	NaN	65000000	['id': 80, 'name': 'Crime'], {'id': 28, 'name...	NaN

2998	2999	NaN	42000000	{'id': 35, 'name': 'Comedy'}, {'id': 10749, '...	http://www.alongcamepolly.com/
2999	3000	NaN	35000000	{'id': 53, 'name': 'Thriller'}, {'id': 28, 'n...	http://www.abductionthefilm.com/

3000 rows × 23 columns

```
df = pd.read_csv('imdb_data.csv')
```

```
# print(df.to_string())
```

```
df
```



	id	belongs_to_collection		budget	genres	homepage
0	1	[[{'id': 313576, 'name': 'Hot Tub Time Machine ...		14000000	[[{'id': 35, 'name': 'Comedy'}]]	NaN
1	2	[[{'id': 107674, 'name': 'The Princess Diaries ...		40000000	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Comedy'}]]	NaN
2	3		NaN	3300000	[[{'id': 18, 'name': 'Drama'}]]	http://sonyclassics.com/whiplash/
3	4		NaN	1200000	[[{'id': 53, 'name': 'Thriller'}, {'id': 18, 'name': 'Drama'}]]	http://kahaanithefilm.com/
4	5		NaN	0	[[{'id': 28, 'name': 'Action'}, {'id': 53, 'name': 'Drama'}]]	NaN
...	...		...	...	...	...
2995	2996		NaN	0	[[{'id': 35, 'name': 'Comedy'}, {'id': 10749, 'name': 'Comedy'}]]	NaN
2996	2997		NaN	0	[[{'id': 18, 'name': 'Drama'}, {'id': 10402, 'name': 'Drama'}]]	NaN
2997	2998		NaN	65000000	[[{'id': 80, 'name': 'Crime'}, {'id': 28, 'name': 'Drama'}]]	NaN

2998	2999	NaN	42000000	{'id': 35, 'name': 'Comedy'}, {'id': 10749, '...	http://www.alongcamepolly.com/
2999	3000	NaN	35000000	{'id': 53, 'name': 'Thriller'}, {'id': 28, 'n...	http://www.abductionthefilm.com/

3000 rows × 23 columns

```
df = pd.read_json('data.json')
```

df

	id	name	age	city	salary	
0	1	Alice	25	New York	70000	
1	2	Bob	30	Los Angeles	85000	
2	3	Charlie	28	Chicago	62000	
3	4	David	35	Houston	91000	
4	5	Eva	22	Phoenix	55000	

Next steps: [Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
data = {
  "Duration":{
    "0":60,
    "1":60,
    "2":60,
    "3":45,
    "4":45,
    "5":60
  },
  "Pulse":{
    "0":110,
    "1":117,
    "2":103,
    "3":109,
    "4":117,
    "5":102
  },
}
```

```

"Maxpulse":{
  "0":130,
  "1":145,
  "2":135,
  "3":175,
  "4":148,
  "5":127
},
"Calories":{
  "0":409,
  "1":479,
  "2":340,
  "3":282,
  "4":406,
  "5":300
}
}

```

```
df = pd.DataFrame(data)
```

```
df
```

	Duration	Pulse	Maxpulse	Calories	
0	60	110	130	409	
1	60	117	145	479	
2	60	103	135	340	
3	45	109	175	282	
4	45	117	148	406	
5	60	102	127	300	

Next steps:

[Generate code with df](#)
[View recommended plots](#)
[New interactive sheet](#)

```
df.head(3)
```

	Duration	Pulse	Maxpulse	Calories	
0	60	110	130	409	
1	60	117	145	479	
2	60	103	135	340	

Next steps:

[Generate code with df](#)
[View recommended plots](#)
[New interactive sheet](#)

df.head()



	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409
1	60	117	145	479
2	60	103	135	340
3	45	109	175	282
4	45	117	148	406



Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

df.tail(3)



	Duration	Pulse	Maxpulse	Calories
3	45	109	175	282
4	45	117	148	406
5	60	102	127	300



df.tail()



	Duration	Pulse	Maxpulse	Calories
1	60	117	145	479
2	60	103	135	340
3	45	109	175	282
4	45	117	148	406
5	60	102	127	300



df.info()



```
<class 'pandas.core.frame.DataFrame'>
Index: 6 entries, 0 to 5
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Duration    6 non-null      int64
1   Pulse       6 non-null      int64
2   Maxpulse    6 non-null      int64
3   Calories    6 non-null      int64
dtypes: int64(4)
```

memory usage: 412.0+ bytes

```
df.describe()
```



	Duration	Pulse	Maxpulse	Calories
count	6.000000	6.000000	6.000000	6.000000
mean	55.000000	109.666667	143.333333	369.333333
std	7.745967	6.501282	17.557525	75.152290
min	45.000000	102.000000	127.000000	282.000000
25%	48.750000	104.500000	131.250000	310.000000
50%	60.000000	109.500000	140.000000	373.000000
75%	60.000000	115.250000	147.250000	408.250000
max	60.000000	117.000000	175.000000	479.000000



```
df.dropna(inplace=True)
```

```
df
```



	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409
1	60	117	145	479
2	60	103	135	340
3	45	109	175	282
4	45	117	148	406
5	60	102	127	300



Next steps:

[Generate code with df](#)

☒ [View recommended plots](#)

[New interactive sheet](#)

```
df.fillna(100000, inplace = True)
```

```
df
```


	Duration	Pulse	Maxpulse	Calories	
0	60	110	130	409	
1	60	117	145	479	
2	60	103	135	340	
3	45	109	175	282	
4	45	117	148	406	
5	60	102	127	300	

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
df.fillna({"Calories": 80000}, inplace=True)
```

df

	Duration	Pulse	Maxpulse	Calories	
0	60	110	130	409	
1	60	117	145	479	
2	60	103	135	340	
3	45	109	175	282	
4	45	117	148	406	
5	60	102	127	300	

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
x = df["Calories"].mean()
```

x

```
 np.float64(369.3333333333333)
```

```
df.fillna({"Calories": x}, inplace=True)
```

```
x = df["Calories"].median()
```

x

```
 373.0
```

```
df.fillna({"Calories": x}, inplace=True)
```

```
x = df["Calories"].mode()
```

```
x
```



Calories

0	282
1	300
2	340
3	406
4	409
5	479

dtype: int64

```
df.fillna({"Calories": x}, inplace=True)
```

```
df.dropna(subset=['Calories'], inplace = True)
```

```
df
```



Duration Pulse Maxpulse Calories

0	60	110	130	409
1	60	117	145	479
2	60	103	135	340
3	45	109	175	282
4	45	117	148	406
5	60	102	127	300



Next steps:

[Generate code with df](#)[View recommended plots](#)[New interactive sheet](#)

```
df.loc[0, 'Calories'] = 45
```

```
df
```



	Duration	Pulse	Maxpulse	Calories	
0	60.0	110.0	130.0	409.0	
1	60.0	117.0	145.0	479.0	
2	60.0	103.0	135.0	340.0	
3	45.0	109.0	175.0	282.0	
4	45.0	117.0	148.0	406.0	
5	60.0	102.0	127.0	300.0	
0	NaN	NaN	NaN	45.0	

Next steps:

[Generate code with df](#)[View recommended plots](#)[New interactive sheet](#)

```
df.dropna(inplace=True)
```

```
df
```



	Duration	Pulse	Maxpulse	Calories
0	60.0	110.0	130.0	120.0
1	60.0	117.0	145.0	120.0
2	60.0	103.0	135.0	120.0
3	45.0	109.0	175.0	120.0
4	45.0	117.0	148.0	120.0
5	60.0	102.0	127.0	120.0

```
for x in df.index:
    if df.loc[x, "Calories"] > 120:
        df.loc[x, "Calories"] = 120
```

```
df
```

	Duration	Pulse	Maxpulse	Calories	
0	60.0	110.0	130.0	120.0	
1	60.0	117.0	145.0	120.0	
2	60.0	103.0	135.0	120.0	
3	45.0	109.0	175.0	120.0	
4	45.0	117.0	148.0	120.0	
5	60.0	102.0	127.0	120.0	

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

```
for x in df.index:
    if df.loc[x, "Maxpulse"] > 140:
        df.drop(x, inplace = True)
```

df

	Duration	Pulse	Maxpulse	Calories	
0	60.0	110.0	130.0	120.0	
2	60.0	103.0	135.0	120.0	
5	60.0	102.0	127.0	120.0	

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

df.duplicated()

	0
0	False
2	False
5	False

dtype: bool

df

	Duration	Pulse	Maxpulse	Calories	
0	60.0	110.0	130.0	120.0	
2	60.0	103.0	135.0	120.0	
5	60.0	102.0	127.0	120.0	

Next steps:

[Generate code with df](#)
[View recommended plots](#)
[New interactive sheet](#)

```
df.drop_duplicates(inplace = True)
```

df

	Duration	Pulse	Maxpulse	Calories	
0	60.0	110.0	130.0	120.0	
2	60.0	103.0	135.0	120.0	
5	60.0	102.0	127.0	120.0	

Next steps:

[Generate code with df](#)
[View recommended plots](#)
[New interactive sheet](#)

```
df.corr()
```

	Duration	Pulse	Maxpulse	Calories	
Duration	NaN	NaN	NaN	NaN	
Pulse	NaN	1.000000	-0.028383	NaN	
Maxpulse	NaN	-0.028383	1.000000	NaN	
Calories	NaN	NaN	NaN	NaN	

```
import pandas as pd
```

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
}, index=['a', 'b', 'c'])
```

```
print(df.loc['a'])
print(df.loc['a', 'Name'])
```

Name	Alice
Age	25

```
Name: a, dtype: object
Alice
```

```
print(df.iloc[0])
print(df.iloc[0, 0])
```

```
⇒ Name    Alice
   Age      25
   Name: a, dtype: object
   Alice
```

```
print(df.loc[['a','c']])
print(df.loc[['a','c'], 'Name'])
```

```
⇒      Name  Age
a    Alice   25
c  Charlie   35
a      Alice
c    Charlie
Name: Name, dtype: object
```

```
print(df.iloc[[0,2]])
print(df.iloc[[0,2], 0])
```

```
⇒      Name  Age
a    Alice   25
c  Charlie   35
a      Alice
c    Charlie
Name: Name, dtype: object
```

```
#read data from csv file
df1 = pd.read_csv('imdb_data.csv')
```

```
df1
```



	id	belongs_to_collection		budget	genres	homepage	
0	1	[[{'id': 313576, 'name': 'Hot Tub Time Machine ...		14000000	[[{'id': 35, 'name': 'Comedy'}]]	NaN	t
1	2	[[{'id': 107674, 'name': 'The Princess Diaries ...		40000000	[[{'id': 35, 'name': 'Comedy'}, {'id': 18, 'name': 'Comedy'}]]	NaN	t
2	3	NaN	NaN	3300000	[[{'id': 18, 'name': 'Drama'}]]	http://sonyclassics.com/whiplash/	t
3	4	NaN	NaN	1200000	[[{'id': 53, 'name': 'Thriller'}, {'id': 18, 'name': 'Drama'}]]	http://kahaanithefilm.com/	t
4	5	NaN	NaN	0	[[{'id': 28, 'name': 'Action'}, {'id': 53, 'name': 'Thriller'}]]	NaN	t
...	...	...	...	...	...	...	...
2995	2996	NaN	NaN	0	[[{'id': 35, 'name': 'Comedy'}, {'id': 10749, 'name': 'Comedy'}]]	NaN	t
2996	2997	NaN	NaN	0	[[{'id': 18, 'name': 'Drama'}, {'id': 10402, 'name': 'Drama'}]]	NaN	t
2997	2998	NaN	NaN	65000000	[[{'id': 80, 'name': 'Crime'}, {'id': 28, 'name': 'Action'}]]	NaN	t

2998	2999	NaN	42000000	{'id': 35, 'name': 'Comedy'}, {'id': 10749, 'n...	http://www.alongcamepolly.com/ t
2999	3000	NaN	35000000	{'id': 53, 'name': 'Thriller'}, {'id': 28, 'n...	http://www.abductionthefilm.com/ t

3000 rows × 23 columns

```
df2 = pd.read_json('data.json')
```

df2



	id	name	age	city	salary	
0	1	Alice	25	New York	70000	
1	2	Bob	30	Los Angeles	85000	
2	3	Charlie	28	Chicago	62000	
3	4	David	35	Houston	91000	
4	5	Eva	22	Phoenix	55000	

Next steps:

Generate code with df2

 View recommended plots

New interactive sheet

```
df2 = pd.read_json('anscombe.json')
```

df2



	Series	X	Y
0	I	10	8.04
1	I	8	6.95
2	I	13	7.58
3	I	9	8.81
4	I	11	8.33
5	I	14	9.96
6	I	6	7.24
7	I	4	4.26
8	I	12	10.84
9	I	7	4.81
10	I	5	5.68
11	II	10	9.14
12	II	8	8.14
13	II	13	8.74
14	II	9	8.77
15	II	11	9.26
16	II	14	8.10
17	II	6	6.13
18	II	4	3.10
19	II	12	9.13
20	II	7	7.26
21	II	5	4.74
22	III	10	7.46
23	III	8	6.77
24	III	13	12.74
25	III	9	7.11
26	III	11	7.81
27	III	14	8.84
28	III	6	6.08
29	III	4	5.39

